Welcome to

Become a Python Developer

Course Instructor: Md. Azizul Hakim

Python Set

Python sets are a versatile and powerful data structure that can store unordered, unique elements.

Features

- Unordered: The elements in a set have no specific order.
- Unique Elements: Sets automatically remove duplicate values.
- Mutable: You can add or remove elements.
- Optimized for Membership Testing: Checking if an element exists in a set is highly efficient (O(1) complexity).

Set Creation

```
# Using curly braces
my_set = {1, 2, 3, 4}
# Using set() constructor
another_set = set([3, 4, 5, 6])
# Empty set
empty_set = set()
```

Basic Operations on Set

1. **Add**
2. **Remove**
3. **Membership**

```
my_set.add(5)
print(my_set)
# Output: {1, 2, 3, 4, 5}
```

```
my_set.remove(2)
my_set.discard(10)  # No error
even if 10 is not in the set
```

```
print(3 in my_set)  # Output: True
print(10 in my_set)  # Output: False
```

More Operations

1. **Union**
2. **Intersection**
3. **Difference**
4. **Symmetric Difference**

Examples

Unique Elements

```
# Input list
numbers = [1, 2, 2, 3, 4, 4, 5]

# Convert to a set to find unique elements
unique_numbers = set(numbers)

print(unique_numbers)  # Output: {1, 2, 3, 4, 5}
```

Common Students

```
# Sets of students
Class_A = {"Alice", "Bob", "Charlie", "David"}
Class_B = {"Charlie", "David", "Eve", "Frank"}

# Find common students
common_students = Class_A & Class_B

print(common_students)  # Output: {'Charlie', 'David'}
```

Remove Duplicates from Two Lists and Merge Them

```
# Input lists
list1 = [1, 2, 3, 3, 4]
list2 = [3, 4, 4, 5, 6]

# Remove duplicates and merge
merged_set = set(list1) | set(list2)

print(merged_set)  # Output: {1, 2, 3, 4, 5, 6}
```

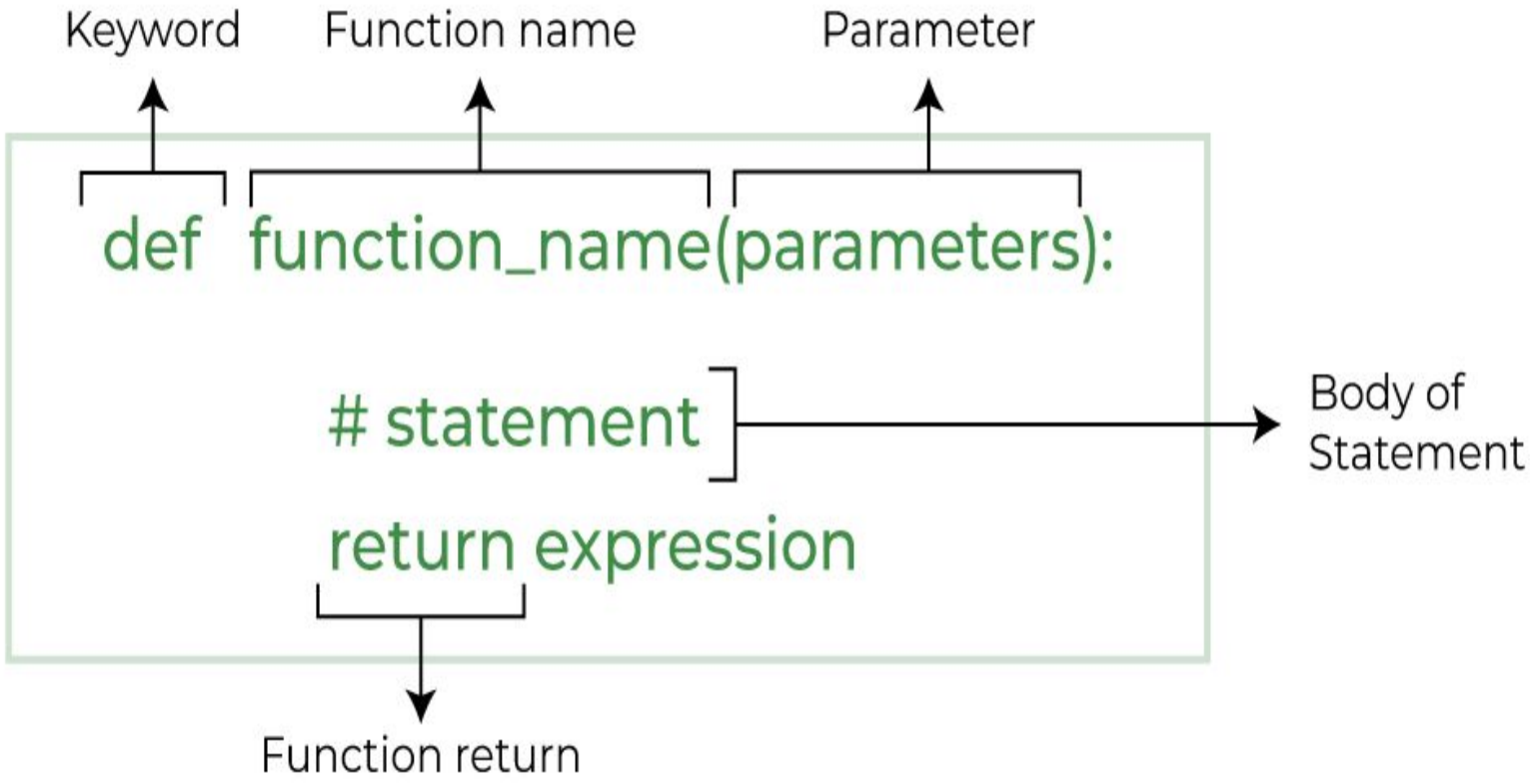
Python Function

a block of statements that return the specific task

Benefits of using function

- **Increase Code Readability**
- **Increase Code Reusability**

Function Syntax



Function types

Types of Function

1. **Standard Library Functions**
2. **User Defined Functions**

Some library functions are

- print()
- pow()
- sqrt()

```
import math

# sqrt computes the square root
square_root = math.sqrt(4)

print("Square Root of 4 is",square_root)

# pow() comptes the power
power = pow(2, 3)

print("2 to the power 3 is",power)
```

User Defined Function

```
def greet():
    print('Hello World!')

# call the function
greet()

print('Outside function')
```

Python Function Arguments

No arguments

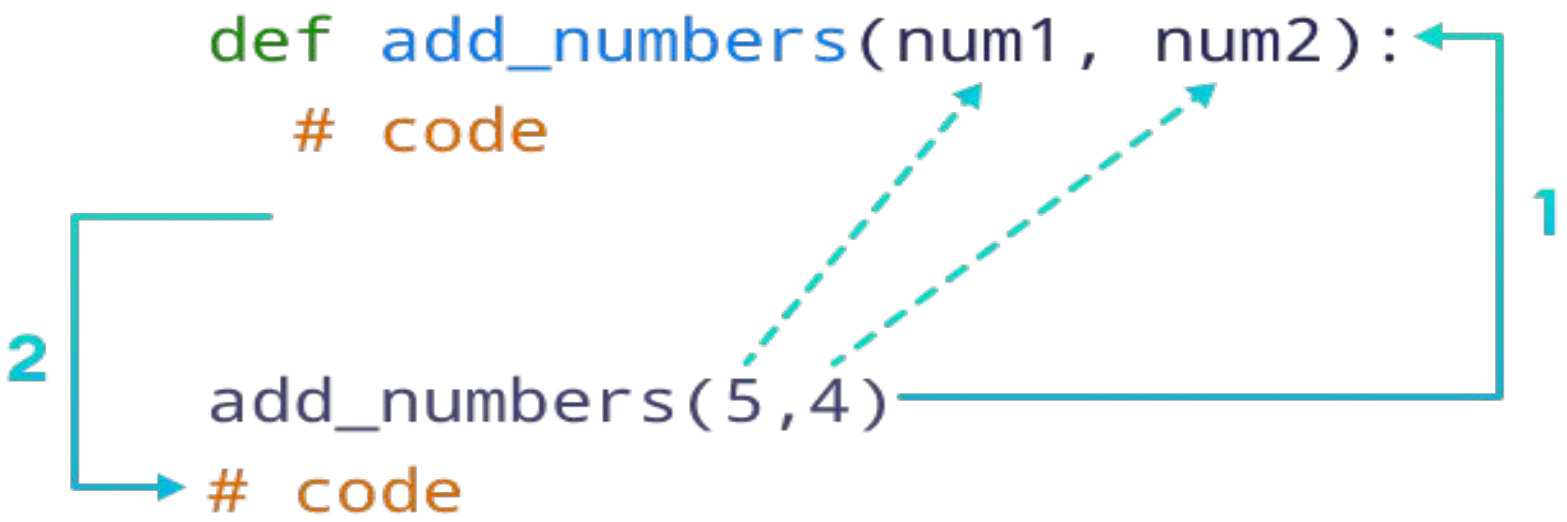
```
def greet():
    print('Hello World!')

# call the function
greet()
```

Two arguments

```
# function with two arguments

def add_numbers(num1, num2):

    sum = num1 + num2
    print('Sum: ',sum))

add_numbers(5,4)
```

Argument Types

- **Default argument**
- **Keyword arguments (named arguments)**
- **Positional arguments**
- **Arbitrary arguments**



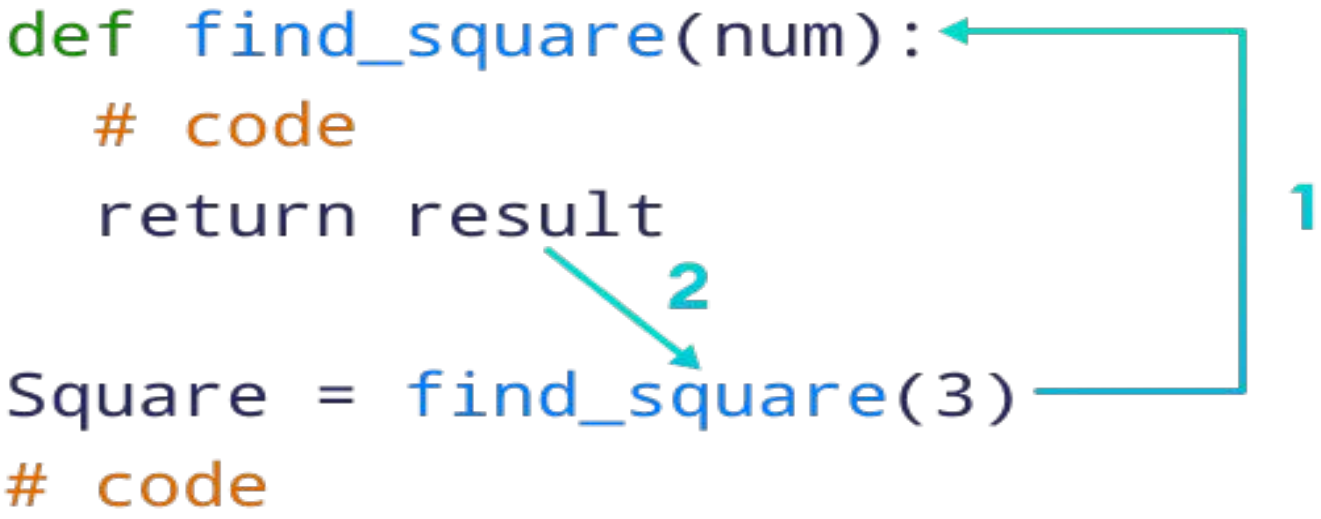
Python Function Return

Return Example

```
def add_numbers():
    ...
    return sum
```

Function return type

```
# function definition
def find_square(num):
    result = num * num
    return result
# function call
square = find_square(3)
print('Square:',square)

# Output: Square: 9
```



Nested Function and Anonymous Function

Function within a function

```
# Python program to
# demonstrate accessing of
# variables of nested functions

def f1():
    s = ‘Ai Quest Python'

    def f2():
        print(s)

    f2()

# Driver's code
f1()
```

Lambda/Anonymous

```
# declare a lambda function
greet = lambda : print('Hello World')

# call lambda function
greet()

# Output: Hello World

# lambda that accepts one argument
greet_user = lambda name : print('Hey there,', name)

# lambda call
greet_user(‘Sanam')

# Output: Hey there, Sanam

#multiple argument
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

Previous class problems using function

Find max elements

```
#function to find max elements of a list
def find_max_element(input_list):
        max_element = input_list[0]
        for number in input_list:
                if number > max_element:
max_element = number
        return max_element

numbers = [7, 2, 9, 4, 1, 5]
result = find_max_element(numbers)
print(result)  # Output: 9
```

Palindrome check

```
#function to check palindrome
def is_palindrome(input_string):
input_string = input_string.lower()
# Convert to lowercase for case insensitivity
reversed_str = input_string[::-1]     return
input_string == reversed_str

text = "racecar“
result = is_palindrome(text)
print(result)  # Output: True
```